

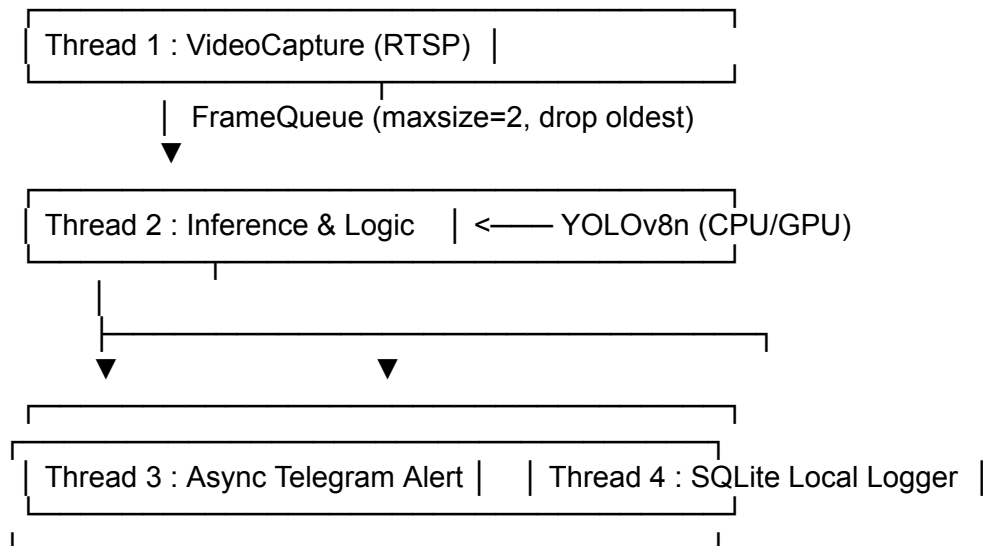


# PACK TECHNIQUE DE GÉNIE LOGICIEL : SENTINEL-EDGE (MVP V0)

Ce dossier technique couvre l'ensemble des livrables de génie logiciel nécessaires pour lancer et exécuter le projet sans dépendance matérielle initiale : architecture système modulaire, code source découplé et threadé, outillage Git/Trello, nomenclature matérielle de projection et documentation technique de référence.

## 1. Architecture Logicielle & Découplage Modulaire

Pour garantir qu'aucun blocage réseau ou calcul d'inférence n'interrompe l'ingestion vidéo, le système adopte une **architecture orientée événements avec files d'attente (Queues) multi-threadées** :



## 2. Configuration Git, GitHub & Workflow Trello

**Arborescence standardisée du dépôt**

```
sentinel-edge/
├── .env.example
├── .gitignore
├── README.md
├── requirements.txt
├── config.py
├── core/
│   ├── __init__.py
│   └── stream.py      # Capture vidéo multi-threadée
```

```

├── detector.py    # Wrapper YOLO & logique anti-faux positifs
├── notifier.py    # Dispatcher asynchrone Telegram
├── database.py    # Repository SQLite
├── app.py         # Orchestrateur principal
├── simu_hardware/ # Espace de travail du Pôle Électronique
│   ├── wokwi/     # Schémas et code C/C++ microcontrôleur
│   └── pcb_kicad/  # Schémas d'alimentation et routage

```

## Fichier .gitignore

```

# Environnements virtuels
venv/
.env

# Python cache & build
__pycache__/_
*.py[cod]
*.pt
*.onnx

# Données locales et captures
captures/
*.db
*.sqlite3

# IDE & OS
.vscode/
.idea/
.DS_Store

```

## Workflow Trello (Colonnes & Tickets Pré-configurés)

- **Backlog Général** : Idées futures (reconnaissance faciale V2, panneau solaire, dashboard cloud).
- **Sprint 1 (Semaines 1–2)** :
  - [IA] Implémenter la classe VideoStream multi-threadée avec reconnexion automatique.
  - [IA] Intégrer l'inférence YOLOv8n avec filtrage temporel (seuil à 0.60 sur person).
  - [RÉSEAU] Créer le bot Telegram via BotFather et implémenter TelegramNotifier.
  - [RÉSEAU] Configurer la base de données SQLite et les tables de logs d'intrusion.
  - [ÉLEC] Modéliser sur Wokwi le schéma ESP32 + capteur PIR + buzzer + relais.
  - [DOC] Rédiger le guide d'installation locale et les tests d'intégration.
- **Sprint 2 (Semaines 3–4)** :
  - [RÉSEAU] Développer l'interface web Flask / FastAPI minimale pour visualiser les captures.
  - [ÉLEC] Concevoir le schéma d'alimentation avec circuit de basculement batterie sur

- KiCAD.
  - [ÉLEC] Modéliser le boîtier 3D sous FreeCAD/Fusion 360.
  - [GLOBAL] Exécuter le test d'endurance de 2 heures et la démo en conditions réelles.
- **En Cours / En Revue / Terminé** : Déplacement des cartes avec assignation nominative.

### 3. Implémentation Logicielle Modulaire (Code Source Prêt à l'Emploi)

#### requirements.txt

```
ultralytics>=8.0.0
opencv-python>=4.8.0
python-dotenv>=1.0.0
requests>=2.31.0
```

#### config.py

```
import os
from dotenv import load_dotenv

load_dotenv()

class Config:
    # 0 pour webcam locale ou URL complète type "http://192.168.1.50:8080/video"
    VIDEO_SOURCE = os.getenv("VIDEO_SOURCE", "0")

    # Paramètres de détection
    CONFIDENCE_THRESHOLD = float(os.getenv("CONFIDENCE_THRESHOLD", "0.60"))
    TARGET_CLASS = 0 # 0 correspond à 'person' dans COCO
    PERSISTENCE_FRAMES = 2 # Détections consécutives requises
    ALERT_COOLDOWN = int(os.getenv("ALERT_COOLDOWN", "10")) # Secondes

    # Telegram
    TELEGRAM_BOT_TOKEN = os.getenv("TELEGRAM_BOT_TOKEN", "")
    TELEGRAM_CHAT_ID = os.getenv("TELEGRAM_CHAT_ID", "")

    # Chemins
    CAPTURE_DIR = "captures"
    DATABASE_PATH = "sentinel.db"
```

#### core/stream.py

```
import cv2
```

```

import threading
import time

class VideoStream:
    """Capture vidéo threadée pour minimiser la latence du flux RTSP/HTTP."""
    def __init__(self, src):
        self.src = int(src) if str(src).isdigit() else src
        self.cap = cv2.VideoCapture(self.src)
        self.ret, self.frame = self.cap.read()
        self.running = True
        self.lock = threading.Lock()

        self.thread = threading.Thread(target=self._update, daemon=True)
        self.thread.start()

    def _update(self):
        while self.running:
            if not self.cap.isOpened():
                time.sleep(1)
                self.cap = cv2.VideoCapture(self.src)
                continue

            ret, frame = self.cap.read()
            if ret:
                with self.lock:
                    self.ret = ret
                    self.frame = frame
            else:
                time.sleep(0.05)

    def read(self):
        with self.lock:
            if self.frame is not None:
                return self.ret, self.frame.copy()
            return False, None

    def release(self):
        self.running = False
        self.thread.join(timeout=1.0)
        self.cap.release()

```

## **core/database.py**

```

import sqlite3
from datetime import datetime

class DatabaseManager:

```

```

"""Gestionnaire de persistance locale des événements d'intrusion."""
def __init__(self, db_path):
    self.db_path = db_path
    self._init_db()

def _init_db(self):
    with sqlite3.connect(self.db_path) as conn:
        cursor = conn.cursor()
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS intrusions (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                timestamp TEXT NOT NULL,
                confidence REAL NOT NULL,
                image_path TEXT NOT NULL
            )
        """)
        conn.commit()

def log_intrusion(self, confidence, image_path):
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with sqlite3.connect(self.db_path) as conn:
        cursor = conn.cursor()
        cursor.execute(
            "INSERT INTO intrusions (timestamp, confidence, image_path) VALUES (?, ?, ?)",
            (timestamp, confidence, image_path)
        )
        conn.commit()

```

## core/notifier.py

```

import threading
import requests

class TelegramNotifier:
    """Gestionnaire asynchrone des alertes Telegram."""
    def __init__(self, token, chat_id):
        self.token = token
        self.chat_id = chat_id
        self.api_url = f"https://api.telegram.org/bot{self.token}"

    def send_alert_async(self, image_path, message):
        if not self.token or not self.chat_id:
            print("[NOTIFIER] Tokens Telegram non configurés. Alerte ignorée.")
            return

        thread = threading.Thread(
            target=self._send_payload,

```

```

        args=(image_path, message),
        daemon=True
    )
    thread.start()

def _send_payload(self, image_path, message):
    try:
        url = f"{self.api_url}/sendPhoto"
        with open(image_path, "rb") as img:
            payload = {"chat_id": self.chat_id, "caption": message}
            files = {"photo": img}
            response = requests.post(url, data=payload, files=files, timeout=10)
            if response.status_code != 200:
                print(f"[NOTIFIER ERREUR] Code {response.status_code}: {response.text}")
    except Exception as e:
        print(f"[NOTIFIER ERREUR] Exception lors de l'envoi : {e}")

```

## core/detector.py

```

import cv2
from ultralytics import YOLO

class IntrusionDetector:
    """Wrapper YOLOv8 pour la détection humaine avec persistance temporelle."""
    def __init__(self, model_name="yolov8n.pt", conf_thresh=0.60, persistence=2):
        self.model = YOLO(model_name)
        self.conf_thresh = conf_thresh
        self.persistence = persistence
        self.consecutive_hits = 0

    def process_frame(self, frame):
        results = self.model(
            frame,
            classes=[0],
            conf=self.conf_thresh,
            verbose=False
        )

        detected = False
        highest_conf = 0.0
        annotated_frame = frame.copy()

        for r in results:
            boxes = r.boxes
            for box in boxes:
                detected = True
                conf = float(box.conf[0])

```

```

        if conf > highest_conf:
            highest_conf = conf

        x1, y1, x2, y2 = map(int, box.xyxy[0])
        cv2.rectangle(annotated_frame, (x1, y1), (x2, y2), (0, 0, 255), 2)
        cv2.putText(
            annotated_frame,
            f"Humain: {conf:.2f}",
            (x1, max(20, y1 - 10)),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.5,
            (0, 0, 255),
            2
        )

    if detected:
        self.consecutive_hits += 1
    else:
        self.consecutive_hits = 0

    # L'intrusion est validée uniquement si la persistance est atteinte
    is_confirmed = self.consecutive_hits >= self.persistence
    return is_confirmed, highest_conf, annotated_frame

```

## app.py

```

import os
import time
import cv2
from datetime import datetime
from config import Config
from core.stream import VideoStream
from core.detector import IntrusionDetector
from core.notifier import TelegramNotifier
from core.database import DatabaseManager

def main():
    os.makedirs(Config.CAPTURE_DIR, exist_ok=True)

    stream = VideoStream(Config.VIDEO_SOURCE)
    detector = IntrusionDetector(
        conf_thresh=Config.CONFIDENCE_THRESHOLD,
        persistence=Config.PERSISTENCE_FRAMES
    )
    notifier = TelegramNotifier(Config.TELEGRAM_BOT_TOKEN, Config.TELEGRAM_CHAT_ID)
    db = DatabaseManager(Config.DATABASE_PATH)

```

```

last_alert_time = 0
print("[SENTINEL-EDGE] Système en écoute. Appuyez sur 'q' pour quitter.")

try:
    while True:
        ret, frame = stream.read()
        if not ret or frame is None:
            time.sleep(0.01)
            continue

        confirmed_intrusion, conf, annotated = detector.process_frame(frame)
        current_time = time.time()

        if confirmed_intrusion and (current_time - last_alert_time >
Config.ALERT_COOLDOWN):
            last_alert_time = current_time
            timestamp_str = datetime.now().strftime("%Y%m%d_%H%M%S")
            filename = os.path.join(Config.CAPTURE_DIR, f"intrusion_{timestamp_str}.jpg")

            cv2.imwrite(filename, annotated)
            db.log_intrusion(conf, filename)

            caption = f"⚠️ ALERTE SENTINEL-EDGE\nIntrusion confirmée\nConfiance :
{conf:.2f}\nDate : {datetime.now().strftime('%d/%m/%Y %H:%M:%S')}"
            notifier.send_alert_async(filename, caption)
            print(f"[ALERTE] Événement horodaté et consigné : {filename}")

            cv2.imshow("Sentinel-Edge Stream", annotated)
            if cv2.waitKey(1) & 0xFF == ord('q'):
                break

    finally:
        stream.release()
        cv2.destroyAllWindows()
        print("[SENTINEL-EDGE] Arrêt propre du système.")

if __name__ == "__main__":
    main()

```

## 4. Nomenclature & Projection des Coûts (Pour le Pôle Électronique)

Pour préparer le passage à la V1 matérielle, voici la grille d'estimation à faire vérifier sur les marchés locaux de Bamako par le binôme électronique :

| Composant               | Référence Recommandée               | Rôle Technique                                 | Fourchette de Prix Estimée   |
|-------------------------|-------------------------------------|------------------------------------------------|------------------------------|
| <b>Micro-ordinateur</b> | Raspberry Pi 4 (2 Go ou 4 Go)       | Calcul local Edge AI et orchestration          | 35 000 – 45 000 F CFA        |
| <b>Capteur Optique</b>  | Pi Camera Module 2 ou 3             | Capture vidéo grand angle 1080p                | 10 000 – 14 000 F CFA        |
| <b>Capteur Présence</b> | Capteur PIR HC-SR501                | Déclenchement physique basse consommation      | 1 000 – 1 500 F CFA          |
| <b>Actionneurs</b>      | Relais 5V 1 voie + Buzzer piézo     | Déclenchement projecteur / alarme sonore       | 1 500 – 2 500 F CFA          |
| <b>Alimentation UPS</b> | Module PowerBank Li-Ion 5V/3A       | Continuité 2 à 4h en cas de coupure de courant | 6 000 – 9 000 F CFA          |
| <b>Connectique</b>      | Câbles Dupont, alimentation secteur | Câblage et liaison GPIO                        | 2 000 – 3 000 F CFA          |
| <b>Total Estimé V1</b>  | —                                   | —                                              | <b>55 500 – 75 000 F CFA</b> |

## 5. Fiche Technique README.md (À commiter à la racine)

#  Sentinel-Edge (SafeHome Africa) — MVP V0

Système d'alerte et de vision intelligente locale (Edge AI) conçu pour les contraintes d'infrastructure locales.

##  Fonctionnalités du MVP

- **Capture Vidéo Découplée** : Supporte smartphone Android (IP Webcam) ou webcam interne.
- **Inférence IA Locale** : Détection humaine via YOLOv8n (CPU/GPU local).
- **Anti-Faux Positifs** : Persistance temporelle de détection et filtrage de confiance.
- **Alertes Téléphone** : Transmission instantanée et asynchrone des photos d'intrusion via Telegram.
- **Journalisation Locale** : Persistance des événements sous SQLite.

##  Installation Rapide

1. Cloner le dépôt et entrer dans le dossier :

```
\\\`bash
git clone <URL_DU_REPO>
cd sentinel-edge
\\\`
```

2. Créer et activer l'environnement virtuel :

```
\\\`bash
python3 -m venv venv
```

```
source venv/bin/activate
```

```
\\`
```

3. Installer les dépendances :

```
\\`bash
```

```
pip install -r requirements.txt
```

```
\\`
```

4. Configurer les variables d'environnement :

Copier `.env.example` en `.env` et renseigner les paramètres du flux vidéo et de Telegram :

```
\\`bash
```

```
cp .env.example .env
```

```
\\`
```

5. Démarrer le système :

```
\\`bash
```

```
python app.py
```

```
\\`
```